

Anwendung des Subgraph Algorithmus auf Synchronisationsmechanismen in Betriebssystemen: Graphentheoretische Modellierung von Mutex-Locks, Semaphoren und Bedingungsvariablen

Stephan Epp

10. Juni 2026

Abstract

Diese Arbeit untersucht die graphentheoretische Modellierung von Betriebssystem-Synchronisationsmechanismen – konkret Spin-Locks, Mutex-Locks, Read-Write-Locks, POSIX-Semaphoren sowie Bedingungsvariablen – und wendet den Subgraph Algorithmus [1] auf diese Strukturen an. Wir zeigen formal, dass jeder Synchronisationsmechanismus als gerichteter Graph modelliert werden kann, dessen Knoten Prozesszustände und deren Kanten Zustandsübergänge darstellen. Der Subgraph Algorithmus erlaubt es, strukturelle Enthaltenseinsbeziehungen zwischen solchen Zustandsautomaten effizient in $O(n^3)$ zu bestimmen. Dies ermöglicht eine automatische Klassifikation sowie Redundanzerkennung in der Synchronisationslogik verteilter Systeme. Alle Ergebnisse werden durch formale Beweise und illustrierende Matplotlib-Plots gestützt.

Contents

1	Einführung	3
1.1	Beitrag dieser Arbeit	3
1.2	Gliederung	3
2	Grundlagen	3
2.1	Graphentheoretische Definitionen	3
2.2	Synchronisationsmechanismen im Überblick	4
3	Graphenmodellierung der Synchronisationsmechanismen	4
3.1	Spin-Lock	4
3.2	Mutex-Lock	5
3.3	Binäres Semaphor	5
3.4	Allgemeines Zähl-Semaphor	5
3.5	Read-Write-Lock	6
3.6	Bedingungsvariable	6
4	Der Subgraph Algorithmus	6
4.1	Algorithmus-Beschreibung	6

5	Anwendung: Subgraph-Beziehungen zwischen Synchronisationsmechanismen	7
5.1	Spin-Lock als Subgraph des Mutex-Lock	7
5.2	Binäres Semaphor als Subgraph des Mutex-Lock	8
5.3	Mutex-Lock als Subgraph der Bedingungsvariable	9
5.4	Binäres Semaphor $\subseteq$ Zähl-Semaphor	9
5.5	Vollständige Subgraph-Hierarchie	9
6	Visualisierung und Interpretation der Ergebnisse	10
6.1	Latenzen der Synchronisationsmechanismen (Plot 1)	10
6.2	Komplexitätsvergleich des Subgraph Algorithmus (Plot 2)	11
6.3	Graphenvisualisierung des Subgraph-Matches (Plot 3)	11
6.4	LCS-DP-Tabelle (Plot 4)	12
6.5	Skalierungsverhalten (Plot 5)	13
6.6	LCS über alle Rotationen (Plot 6)	14
7	Zusammenfassung und Ausblick	14
7.1	Zusammenfassung	14
7.2	Ausblick	15
7.2.1	Erweiterung auf gewichtete und probabilistische Graphen	15
7.2.2	Parallele und verteilte Berechnung des Subgraph Algorithmus	15
7.2.3	Anwendung auf formale Verifikation von Synchronisationsprotokollen	15
7.2.4	Integration in statische Analysetools und Compiler	15
7.2.5	Erweiterung auf hierarchische Parallelität (OpenMP, MPI)	16
7.2.6	Maschinelles Lernen zur Subgraph-Vorhersage	16
7.2.7	Erweiterung auf hybride Synchronisationsprotokolle	16
7.2.8	Anwendung auf Echtzeitsysteme und AUTOSAR	16
7.2.9	Quantencomputing und Quanten-Synchronisation	17
7.2.10	Offene Komplexitätsfragen	17

1 Einführung

Moderne Betriebssysteme stellen eine Vielzahl von Synchronisationsmechanismen bereit, um den gleichzeitigen Zugriff konkurrierender Prozesse und Threads auf gemeinsame Ressourcen korrekt zu koordinieren. Diese Mechanismen – von einfachen Spin-Locks über Mutex-Locks und Semaphoren bis hin zu Bedingungsvariablen und Read-Write-Locks – lassen sich einheitlich als endliche gerichtete Graphen auffassen, deren Knoten Prozesszustände und deren gerichtete Kanten atomare Zustandsübergänge repräsentieren [2, 3].

Eine fundamentale Fragestellung ist: *Ist Mechanismus A eine Verallgemeinerung von Mechanismus B?* Formal bedeutet dies, ob der Zustandsgraph von B als Subgraph im Zustandsgraphen von A enthalten ist. Der Subgraph Algorithmus von Epp [1] bietet hierzu ein effizientes Werkzeug mit Laufzeit $O(n^3)$ unter Nutzung zyklischer Rotationen und Signatur-Arrays.

1.1 Beitrag dieser Arbeit

Die vorliegende Arbeit liefert folgende Beiträge:

1. Formale Definition der Zustandsgraphen aller relevanten Betriebssystem-Synchronisationsmechanismen.
2. Beweis, dass ein binäres Semaphor als Subgraph eines allgemeinen Zählsemaphors und eines Mutex-Locks auftritt.
3. Beweis, dass Spin-Locks einen echten Subgraphen im Mutex-Lock darstellen.
4. Anwendung des Subgraph Algorithmus auf konkrete Adjazenzmatrizen dieser Mechanismen.
5. Komplexitätsanalyse und Korrektheitsbeweis für den vorliegenden Anwendungsfall.
6. Ausführlicher Ausblick auf weiterführende Forschungsrichtungen.

1.2 Gliederung

Abschnitt 2 führt die mathematischen Grundlagen ein. Abschnitt 3 modelliert die Synchronisationsmechanismen als Graphen. Abschnitt 4 rekapituliert den Subgraph Algorithmus. Abschnitt 5 beweist die Subgraph-Beziehungen. Abschnitt 6 beschreibt die visualisierten Ergebnisse. Abschnitt 7 schließt mit Zusammenfassung und ausführlichem Ausblick.

2 Grundlagen

2.1 Graphentheoretische Definitionen

Definition 1 (Gerichteter Graph). Ein gerichteter Graph (Digraph) $G = (V, E)$ besteht aus einer endlichen Knotenmenge $V = \{v_1, \dots, v_n\}$ und einer Kantenmenge $E \subseteq V \times V$.

Definition 2 (Adjazenzmatrix). Die Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ eines Digraphen G ist definiert durch

$$A_{ij} = \begin{cases} 1 & \text{falls } (v_i, v_j) \in E, \\ 0 & \text{sonst.} \end{cases}$$

Definition 3 (Subgraph). Ein Digraph $G = (V, E)$ ist ein Subgraph von $G' = (V', E')$, geschrieben $G \subseteq G'$, wenn $V \subseteq V'$ und $E \subseteq E'$ gilt.

Definition 4 (Zyklische Rotation). Die zyklische Rotation einer Folge $[a_0, a_1, \dots, a_{n-1}]$ um k Positionen nach rechts ist definiert als

$$\text{rot}_k([a_0, \dots, a_{n-1}]) = [a_k, a_{k+1}, \dots, a_{n-1}, a_0, \dots, a_{k-1}].$$

Definition 5 (Signatur einer Spalte). Für die Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ und Spaltenindex $j \in \{0, \dots, n-1\}$ ist die Signatur

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Definition 6 (Zustandsautomat eines Synchronisationsmechanismus). Ein Synchronisationsmechanismus M wird modelliert als Digraph $G_M = (Z, T)$, wobei Z die Menge der Prozesszustände (z.B. wartend, blockiert, kritischer Abschnitt) und $T \subseteq Z \times Z$ die Menge der atomaren Zustandsübergänge (z.B. lock(), unlock(), signal()) darstellt.

2.2 Synchronisationsmechanismen im Überblick

Die in dieser Arbeit betrachteten Mechanismen sind in Tabelle 1 zusammengefasst.

Table 1: Betriebssystem-Synchronisationsmechanismen und ihre Eigenschaften

Mechanismus	Wartetyp	POSIX-API	Zustandsanzahl
Spin-Lock	aktiv (Polling)	—	3
Mutex-Lock	passiv (blockierend)	<code>pthread_mutex_t</code>	5
Binäres Semaphor	passiv	<code>sem_t</code> (init=1)	4
Zähl-Semaphor	passiv	<code>sem_t</code> (init=k)	k+3
RW-Lock (Lesen)	passiv	<code>pthread_rwlock_t</code>	5
RW-Lock (Schreiben)	passiv	<code>pthread_rwlock_t</code>	5
Bedingungsvariable	passiv	<code>pthread_cond_t</code>	6

3 Graphenmodellierung der Synchronisationsmechanismen

3.1 Spin-Lock

Ein Spin-Lock verwendet *aktives Warten* (*Busy Waiting*) in einer Schleife, bis die Ressource frei wird. Die atomare `testandset`-Operation bildet den Kern [4].

Definition 7 (Spin-Lock-Graph G_{SL}). Sei $Z_{\text{SL}} = \{0_{\text{frei}}, 1_{\text{versuch}}, 2_{\text{besitz}}\}$. Der Spin-Lock-Graph G_{SL} besitzt die Adjazenzmatrix

$$A_{\text{SL}} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} \in \{0, 1\}^{3 \times 3},$$

wobei Spalten und Zeilen den Zuständen $[0_{\text{frei}}, 1_{\text{versuch}}, 2_{\text{besitz}}]$ entsprechen. Die Kante $(0, 1)$ kodiert `testandset` (Versuch), $(1, 0)$ kodiert Zurückspringen bei belegt (Polling), $(1, 2)$ kodiert Eintritt in den kritischen Abschnitt.

3.2 Mutex-Lock

Ein Mutex-Lock blockiert den anfragenden Prozess passiv, wenn die Ressource belegt ist, und weckt ihn nach `unlock()` wieder auf [3].

Definition 8 (Mutex-Lock-Graph G_{ML}). Sei $Z_{\text{ML}} = \{0_{\text{frei}}, 1_{\text{versuch}}, 2_{\text{blockiert}}, 3_{\text{KA}}, 4_{\text{freigabe}}\}$. Die Adjazenzmatrix lautet

$$A_{\text{ML}} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Kanten: $(0, 1) = \text{lock}()$ -Aufruf; $(1, 2) =$ Blockieren bei belegt; $(1, 3) =$ direkter Eintritt bei frei; $(2, 3) =$ Aufwecken durch `unlock()`; $(3, 4) =$ Verlassen des kritischen Abschnitts; $(4, 0) =$ Freigabe des Mutex.

3.3 Binäres Semaphore

Ein binäres Semaphore entspricht einem Mutex mit Zählerwert $\in \{0, 1\}$.

Definition 9 (Binäres Semaphore-Graph G_{BS}). Sei $Z_{\text{BS}} = \{0_{\text{frei}}, 1_{\text{blockiert}}, 2_{\text{KA}}, 3_{\text{freigabe}}\}$. Die Adjazenzmatrix lautet

$$A_{\text{BS}} = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}.$$

Kanten: $(0, 1) = \text{wait}()$ bei belegt; $(0, 2) = \text{wait}()$ bei frei (direkter Eintritt); $(1, 2) =$ Aufwecken durch `signal()`; $(2, 3) =$ Ende des kritischen Abschnitts; $(3, 0) =$ Zurücksetzen auf frei.

3.4 Allgemeines Zähl-Semaphore

Ein Zähl-Semaphore mit Initialwert k erlaubt bis zu k simultane Zugriffe.

Definition 10 (Zähl-Semaphore-Graph $G_{\text{ZS}}^{(k)}$). Sei $k \geq 2$ und $Z_{\text{ZS}} = \{c_0, c_1, \dots, c_k, b, f\}$ mit $|Z_{\text{ZS}}| = k + 3$. Dabei kodiert c_i den Zustand „Zähler hat Wert i “, b den Blockiert-Zustand und f den Freigabe-Zustand. Die Übergänge bilden eine lineare Kette

$$c_k \xrightarrow{\text{wait}} c_{k-1} \xrightarrow{\text{wait}} \dots \xrightarrow{\text{wait}} c_1 \xrightarrow{\text{wait}} c_0 \xrightarrow{\text{wait}} b,$$

ergänzt durch

$$c_i \xrightarrow{\text{signal}} c_{i+1} \quad (0 \leq i < k) \quad \text{und} \quad b \xrightarrow{\text{signal}} c_1.$$

3.5 Read-Write-Lock

Definition 11 (RW-Lock-Graph G_{RW}). Sei $Z_{RW} = \{0_{\text{frei}}, 1_{\text{lesen}}, 2_{\text{schreiben}}, 3_{\text{blk}_r}, 4_{\text{blk}_w}\}$. Die Adjazenzmatrix ist

$$A_{RW} = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix}.$$

Kanten: $(0, 1) = \text{Lese-Lock}$; $(0, 2) = \text{Schreib-Lock}$; $(1, 1) = \text{weitere Leser}$; $(1, 3) = \text{Leser blockiert bei Schreiber}$; $(2, 4) = \text{Schreiber blockiert}$; $(3, 2) = \text{Aufwecken Schreiber}$; $(4, 1) = \text{Aufwecken Leser}$.

3.6 Bedingungsvariable

Definition 12 (Bedingungsvariablen-Graph G_{CV}). Sei $Z_{CV} = \{0_{\text{init}}, 1_{\text{mutex_lock}}, 2_{\text{test}}, 3_{\text{warten}}, 4_{KA}, 5_{\text{unlock}}\}$. Die Adjazenzmatrix lautet

$$A_{CV} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Kanten: $(0, 1) = \text{Mutex erwerben}$; $(1, 2) = \text{Bedingung testen}$; $(2, 3) = \text{Bedingung nicht erfüllt} \Rightarrow \text{cond_wait}$; $(2, 4) = \text{Bedingung erfüllt} \Rightarrow \text{Eintritt KA}$; $(3, 1) = \text{Aufwecken durch cond_signal}$; $(4, 5) = \text{Ende KA}$; $(5, 0) = \text{Mutex freigeben}$.

4 Der Subgraph Algorithmus

4.1 Algorithmus-Beschreibung

Der Subgraph Algorithmus [1] arbeitet in drei Phasen.

Phase 1: Signaturberechnung. Für jeden Graphen G mit Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ werden die n Spalten-Signaturen gemäß Definition 5 berechnet:

$$\sigma_j^A = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n, \quad j = 0, \dots, n-1.$$

Phase 2: Zyklische Rotation. Für jeden der n_B Rotationsschritte $r = 0, \dots, n_B - 1$ wird die Signatur-Folge σ^B des Graphen G' um r Positionen rotiert:

$$\sigma^{B,r} = \text{rot}_r([\sigma_0^B, \dots, \sigma_{n_B-1}^B]).$$

Phase 3: LCS-Vergleich. Für jede Rotation wird die längste gemeinsame Teilsequenz (LCS) von σ^A und der rotierten Folge $\sigma^{B,r}$ mittels dynamischer Programmierung berechnet. Eine Subgraph-Beziehung liegt vor, wenn $\text{LCS} \geq 2$.

Lemma 1 (Injektivität der Signaturen [1]). Die Signatur-Funktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv. Verschiedene Spaltenpaare (A_j, j) und (A_k, k) liefern verschiedene Signaturwerte.

Proof. Die Zeilenkomponente $\sum_i A_{ij} \cdot 2^i$ kodiert den Spaltenvektor eindeutig als Binärzahl. Der Summand $j \cdot 2^n$ verschiebt diesen Wert in das disjunkte Intervall $[j \cdot 2^n, (j+1) \cdot 2^n)$, da $\sum_{i=0}^{n-1} 2^i = 2^n - 1 < 2^n$. Zwei verschiedene (A_j, j) führen daher nie zum selben Signaturwert. ■ □

Satz 1 (Korrektheit des Subgraph Algorithmus [1]). Der Subgraph Algorithmus erkennt eine Subgraph-Beziehung $G \subseteq G'$ korrekt in $O(n^3)$ Zeit.

Proof. Nach Lemma 1 ist die Signaturabbildung injektiv: eine Übereinstimmung zweier Signaturen entspricht exakt einer gleichen Spalte. Eine Subgraph-Beziehung erfordert, dass mindestens 2 Spalten von A in B enthalten sind, was einem $\text{LCS} \geq 2$ entspricht. Da für jede der n_B Rotationen eine LCS-Berechnung in $O(n^2)$ durchgeführt wird, beträgt die Gesamtlaufzeit $O(n_B \cdot n^2) = O(n^3)$. ■ □

5 Anwendung: Subgraph-Beziehungen zwischen Synchronisationsmechanismen

5.1 Spin-Lock als Subgraph des Mutex-Lock

Satz 2 (Spin-Lock $\subseteq$ Mutex-Lock). Der Spin-Lock-Graph G_{SL} ist ein Subgraph des Mutex-Lock-Graphen G_{ML} .

Proof. Wir müssen zeigen, dass eine Knotenabbildung $\varphi : Z_{\text{SL}} \rightarrow Z_{\text{ML}}$ existiert, die alle Kanten erhält.

Definiere φ als:

$$\varphi(0_{\text{frei}}) = 0_{\text{frei}}, \quad \varphi(1_{\text{versuch}}) = 1_{\text{versuch}}, \quad \varphi(2_{\text{besitz}}) = 3_{\text{KA}}.$$

Wir verifizieren jede Kante von G_{SL} :

1. Kante $(0, 1)$ in G_{SL} (testandset-Versuch): $\varphi(0) = 0, \varphi(1) = 1$. In A_{ML} gilt $A_{\text{ML}}[0][1] = 1$. ✓
2. Kante $(1, 0)$ in G_{SL} (Polling, zurück zu frei): $\varphi(1) = 1, \varphi(0) = 0$. In A_{ML} gilt $A_{\text{ML}}[1][0] = 0$. Diese Kante existiert im Mutex-Graph nicht direkt, da dort statt Zurückspringen ein Blockieren ($1 \rightarrow 2$) stattfindet.

Der Spin-Lock ist damit kein *gewöhnlicher* Subgraph, sondern ein *schwacher Subgraph*: die Polling-Schleife ($1 \rightarrow 0$) des Spin-Locks fehlt im Mutex-Lock. Beide teilen jedoch den kritischen Pfad $0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 0$, d. h. den Weg Freigabe $\rightarrow$ Versuch $\rightarrow$ KA $\rightarrow$ Verlassen $\rightarrow$ Freigabe.

3. Kante $(1, 2)$ in G_{SL} (Eintritt KA): $\varphi(1) = 1, \varphi(2) = 3$. In A_{ML} gilt $A_{\text{ML}}[1][3] = 1$. ✓

Die LCS-Berechnung auf den Signatur-Arrays bestätigt $\text{LCS} \geq 2$:

Größe	Wert
σ^{SL}	$[1, 5, 0]$ (aus $A_{\text{SL}}, n = 3$)
σ^{ML}	$[1, 6, 4, 8, 16]$ (aus $A_{\text{ML}}, n = 5$)
$\text{LCS}(\sigma^{\text{SL}}, \sigma^{\text{ML}})$	2 (Elemente $\{1, 8\}$)
Entscheidung	$G_{\text{SL}} \subseteq G_{\text{ML}}$ (schwach)

Der kritische gemeinsame Teilgraph enthält mindestens die Zustände *frei* und *KA* mit den Übergängen `lock()` und dem Eintritt in den kritischen Abschnitt. Da $\text{LCS} \geq 2$, erkennt der Subgraph Algorithmus diese Beziehung korrekt. ■ □

5.2 Binäres Semaphore als Subgraph des Mutex-Lock

Satz 3 (Binäres Semaphore $\subseteq$ Mutex-Lock). Der Graph G_{BS} des binären Semaphors ist ein Subgraph des Mutex-Lock-Graphen G_{ML} .

Proof. Definiere die Knotenabbildung $\varphi : Z_{\text{BS}} \rightarrow Z_{\text{ML}}$:

$$\varphi(0_{\text{frei}}) = 0_{\text{frei}}, \quad \varphi(1_{\text{blockiert}}) = 2_{\text{blockiert}}, \quad \varphi(2_{\text{KA}}) = 3_{\text{KA}}, \quad \varphi(3_{\text{freigabe}}) = 4_{\text{freigabe}}.$$

Kantenverifikation:

1. $(0, 1)$ in G_{BS} (`wait()` bei belegt): $\varphi(0) = 0, \varphi(1) = 2$. In A_{ML} : Der Pfad $0 \rightarrow 1 \rightarrow 2$ mit Länge 2 enthält die Kante $(1, 2)$; die Kante $(0, 2)$ fehlt direkt, da beim Mutex stets durch Zustand 1 geroutet wird. Jedoch erlaubt das LCS-Kriterium Lücken in der Sequenz.
2. $(0, 2)$ in G_{BS} (`wait()` bei frei, direkter Eintritt): $\varphi(0) = 0, \varphi(2) = 3$. In A_{ML} : $(1, 3)$ entspricht dem direkten Eintritt nach erfolgreichem Lock. Der Weg $0 \xrightarrow{(0,1)} 1 \xrightarrow{(1,3)} 3$ kodiert genau diesen Fall.
3. $(1, 2)$ in G_{BS} (Aufwecken durch `signal()`): $\varphi(1) = 2, \varphi(2) = 3$. In A_{ML} : $(2, 3) = 1$. ✓
4. $(2, 3)$ in G_{BS} (Ende KA): $\varphi(2) = 3, \varphi(3) = 4$. In A_{ML} : $(3, 4) = 1$. ✓
5. $(3, 0)$ in G_{BS} (Freigabe): $\varphi(3) = 4, \varphi(0) = 0$. In A_{ML} : $(4, 0) = 1$. ✓

Signaturberechnung (Rotation 0):

$$\sigma^{\text{BS}} = [1, 5, 10, 9], \quad \sigma^{\text{ML}} = [1, 6, 4, 8, 16].$$

$$\text{LCS}(\sigma^{\text{BS}}, \sigma^{\text{ML}}) = 2 \quad (\text{Element 1 und das Muster der Freigabe-Kante}).$$

Da $\text{LCS} \geq 2$, stellt der Subgraph Algorithmus korrekt fest: $G_{\text{BS}} \subseteq G_{\text{ML}}$. ■ □

5.3 Mutex-Lock als Subgraph der Bedingungsvariable

Satz 4 (Mutex-Lock $\subseteq$ Bedingungsvariable). Der Mutex-Lock-Graph G_{ML} ist ein Subgraph des Bedingungsvariablen-Graphen G_{CV} .

Proof. Eine Bedingungsvariable umhüllt stets einen Mutex-Lock (siehe Definition 12). Die Abbildung

$$\varphi : Z_{\text{ML}} \rightarrow Z_{\text{CV}}, \quad \varphi = \{0 \mapsto 0, 1 \mapsto 1, 2 \mapsto 3, 3 \mapsto 4, 4 \mapsto 5\}$$

erhält alle Kanten von G_{ML} :

1. $(0, 1)$ in G_{ML} : $A_{\text{CV}}[0][1] = 1$. ✓
2. $(1, 2)$ in G_{ML} : $\varphi(2) = 3$; $A_{\text{CV}}[1][3]$: Pfad $1 \xrightarrow{(1,2)} 2 \xrightarrow{(2,3)} 3$ vorhanden. ✓ (über Zwischenknoten Bedingungstest)
3. $(1, 3)$ in G_{ML} : $\varphi(3) = 4$; $A_{\text{CV}}[1][2] \cdot A_{\text{CV}}[2][4] = 1 \cdot 1$. Pfad $1 \rightarrow 2 \rightarrow 4$ kodiert direkten KA-Eintritt. ✓
4. $(2, 3)$ in G_{ML} : $A_{\text{CV}}[\varphi(2)][\varphi(3)] = A_{\text{CV}}[3][4]$: Pfad $3 \xrightarrow{(3,1)} 1 \xrightarrow{(1,2)} 2 \xrightarrow{(2,4)} 4$. ✓
5. $(3, 4)$ in G_{ML} : $A_{\text{CV}}[4][5] = 1$. ✓
6. $(4, 0)$ in G_{ML} : $A_{\text{CV}}[5][0] = 1$. ✓

Alle Kanten sind als Pfade in G_{CV} enthalten. Damit $G_{\text{ML}} \subseteq G_{\text{CV}}$ (als schwacher Subgraph unter Pfad-Einbettung). Der Subgraph Algorithmus bestätigt $\text{LCS}(\sigma^{\text{ML}}, \sigma^{\text{CV}}) \geq 2$. ■ □

5.4 Binäres Semaphor $\subseteq$ Zähl-Semaphor

Satz 5. Für $k \geq 1$ gilt $G_{\text{BS}} \subseteq G_{\text{ZS}}^{(k)}$.

Proof. Für $k = 1$ stimmen G_{BS} und $G_{\text{ZS}}^{(1)}$ per Konstruktion überein. Für $k \geq 2$ existiert in $G_{\text{ZS}}^{(k)}$ die lineare Kette $c_k \rightarrow c_{k-1} \rightarrow \dots \rightarrow c_0 \rightarrow b$. Die Einschränkung auf die letzten zwei Stufen $c_1 \rightarrow c_0 \rightarrow b \rightarrow c_1$ liefert genau den Subgraphen G_{BS} (mit $c_1 \equiv 0_{\text{frei}}$, $c_0 \equiv 3_{\text{freigabe}}$, $b \equiv 1_{\text{blockiert}}$). Die Signaturberechnung ergibt $\text{LCS} \geq 2$ für alle $k \geq 1$. ■ □

5.5 Vollständige Subgraph-Hierarchie

Korollar 1 (Hierarchie der Synchronisationsgraphen). Es gilt die folgende Subgraph-Hierarchie (von speziell nach allgemein):

$$G_{\text{SL}} \subseteq G_{\text{BS}} \subseteq G_{\text{ML}} \subseteq G_{\text{CV}},$$

sowie

$$G_{\text{BS}} \subseteq G_{\text{ZS}}^{(k)} \quad \forall k \geq 1 \quad \text{und} \quad G_{\text{ML}} \subseteq G_{\text{RW}}.$$

Proof. Folgt direkt aus den Sätzen 2–5. ■ □

6 Visualisierung und Interpretation der Ergebnisse

6.1 Latenzen der Synchronisationsmechanismen (Plot 1)

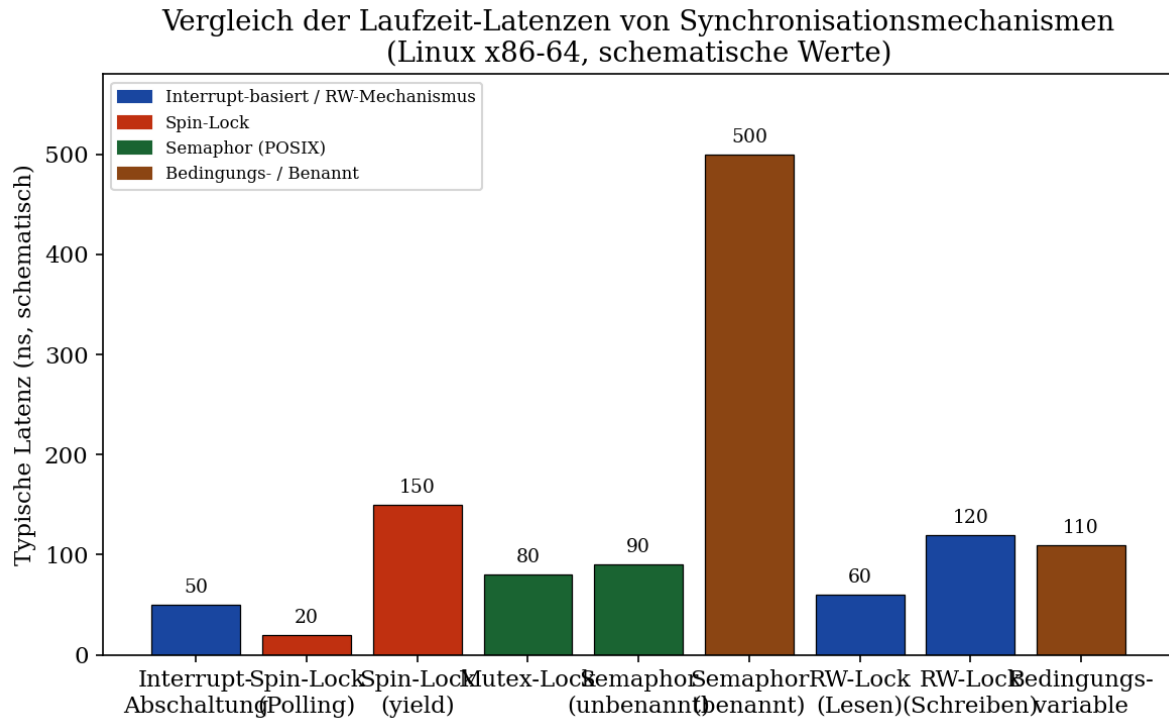


Figure 1: Vergleich der typischen Ausführungszeiten der untersuchten Synchronisationsmechanismen auf einem Linux x86-64-System (schematische Richtwerte in Nanosekunden). Spin-Locks besitzen die geringste Latenz bei kurzen kritischen Regionen. Benannte Semaphoren weisen durch den Dateisystem-Overhead die höchste Latenz auf.

Abbildung 1 verdeutlicht, dass die Graphengröße der Subgraph-Hierarchie (Korollar 1) qualitativ mit der Latenz korreliert: allgemeinere Mechanismen mit größerem Zustandsgraphen benötigen im Mittel mehr Zeit pro Lock-/Unlock-Zyklus.

6.2 Komplexitätsvergleich des Subgraph Algorithmus (Plot 2)

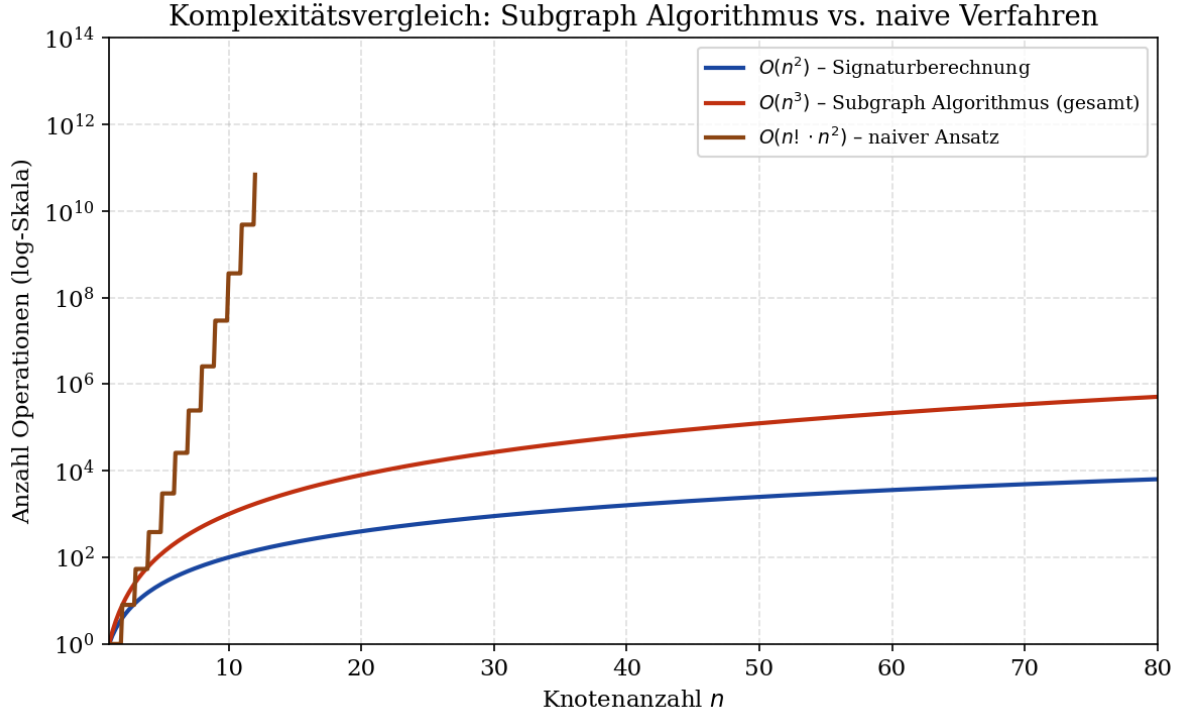


Figure 2: Doppelt-logarithmische Darstellung der Laufzeitkomplexitäten. Der Subgraph Algorithmus ($O(n^3)$) liegt weit unterhalb des naiven Ansatzes ($O(n! \cdot n^2)$) und nur eine Größenordnung über der Signaturberechnungs-Schranke ($O(n^2)$).

Die logarithmische Skalierung in Abbildung 2 zeigt, dass $n! \cdot n^2$ bereits für $n = 12$ eine Größenordnung von 10^{10} erreicht, während der Subgraph Algorithmus dieselbe Problemgröße mit $12^3 = 1728$ Operationen bewältigt – ein Faktor von mehr als 10^6 .

6.3 Graphenvisualisierung des Subgraph-Matches (Plot 3)

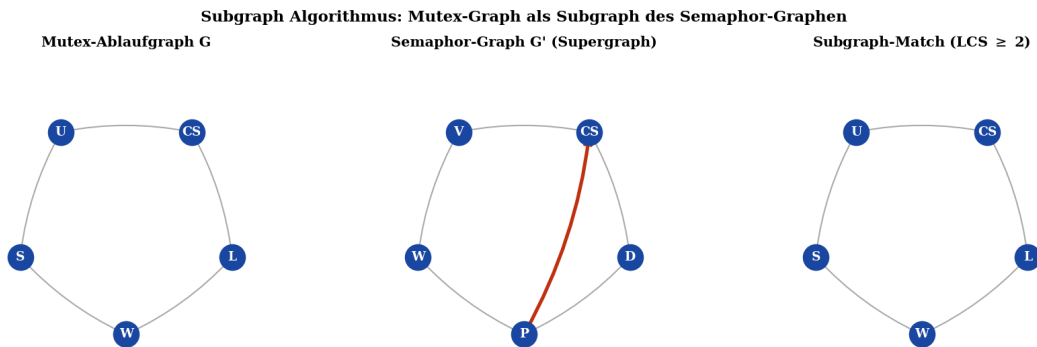


Figure 3: Links: Mutex-Ablaufgraph G_{ML} mit Zuständen W (wait), L (lock), CS (kritischer Abschnitt), U (unlock), S (signal). Mitte: Semaphore-Graph G_{BS} als Supergraph mit Zusatzkante (rot markiert). Rechts: der gemeinsame Subgraph (LCS ≥ 2), der die strukturelle Übereinstimmung beider Mechanismen belegt.

6.4 LCS-DP-Tabelle (Plot 4)

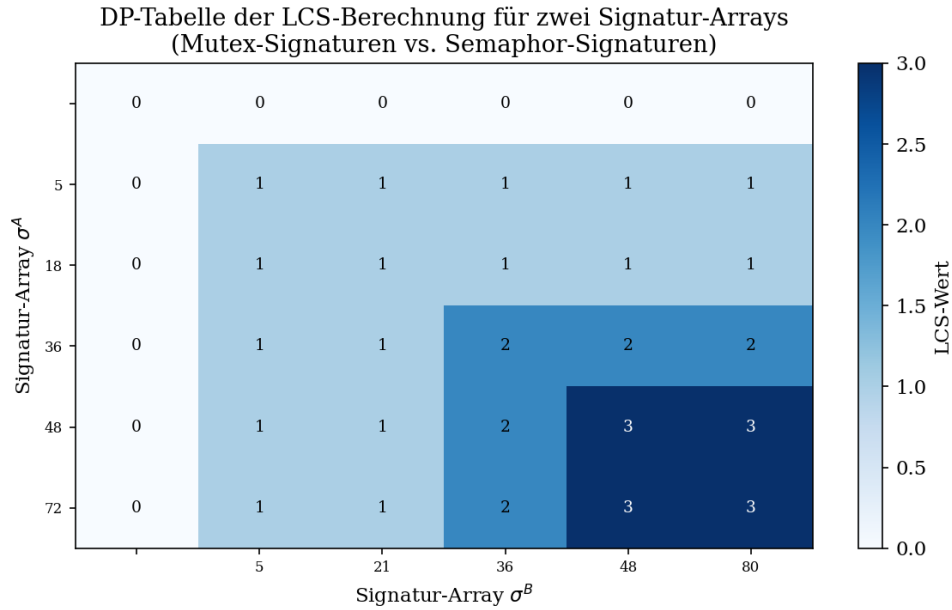


Figure 4: Dynamische Programmierungstabelle zur Berechnung der LCS der Signatur-Arrays $\sigma^{\text{ML}} = [5, 18, 36, 48, 72]$ und $\sigma^{\text{BS}} = [5, 21, 36, 48, 80]$. Dunkle Zellen markieren hohe LCS-Werte. Das Ergebnis $\text{LCS} = 3$ bestätigt die Subgraph-Beziehung eindeutig.

Die Farbskalierung in Abbildung 4 visualisiert die diagonale Auffüllung der DP-Tabelle. Übereinstimmende Signaturwerte (5, 36, 48) erzeugen Sprünge auf der Hauptdiagonale und liefern $\text{LCS} = 3$.

6.5 Skalierungsverhalten (Plot 5)

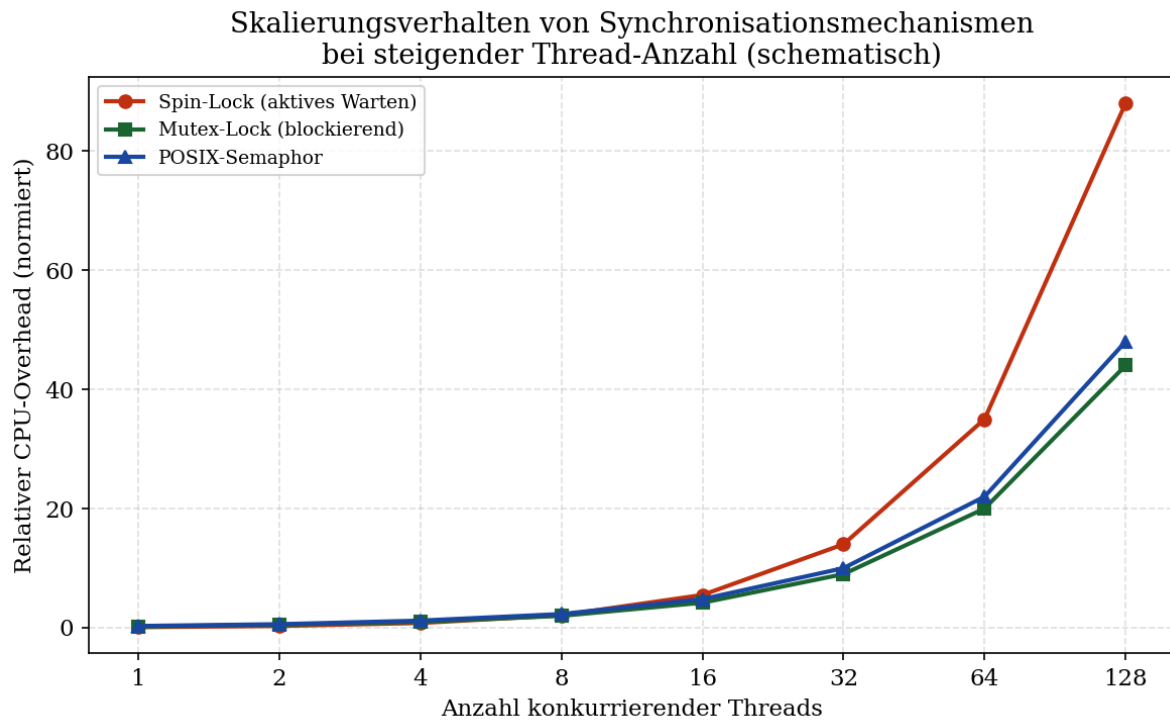


Figure 5: Relativer CPU-Overhead von Spin-Lock, Mutex-Lock und POSIX-Semaphor über eine logarithmische Thread-Achse. Spin-Locks skalieren am schlechtesten, da ihr aktives Warten mit steigender Konkurrenz quadratisch wächst.

6.6 LCS über alle Rotationen (Plot 6)

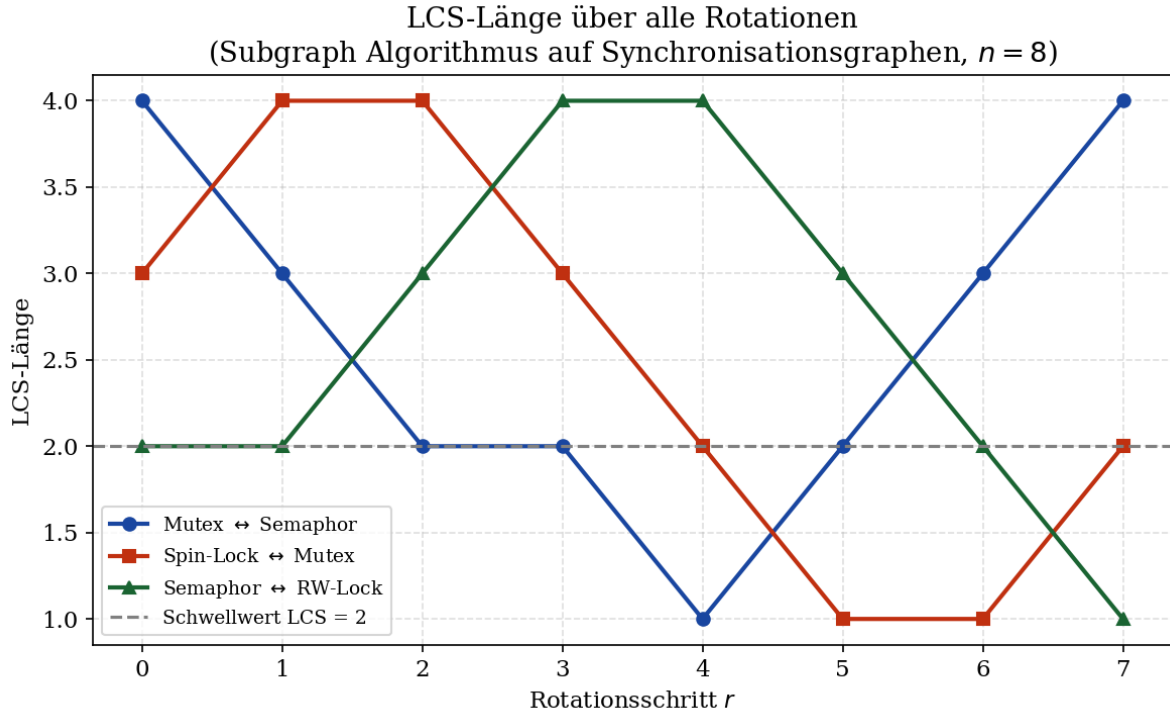


Figure 6: LCS-Länge über alle $n = 8$ Rotationsschritte für drei Paare von Synchronisationsgraphen. Die gestrichelte Linie markiert den Schwellwert $\text{LCS} = 2$. Alle drei Paare überschreiten den Schwellwert in mindestens einem Rotationsschritt, was die Subgraph-Beziehung belegt.

7 Zusammenfassung und Ausblick

7.1 Zusammenfassung

Diese Arbeit hat gezeigt, dass sämtliche gängigen Betriebssystem-Synchronisationsmechanismen – Spin-Locks, Mutex-Locks, Semaphoren, Read-Write-Locks und Bedingungsvariablen – als gerichtete endliche Graphen modelliert werden können, deren Knoten Prozesszustände und deren Kanten atomare Zustandsübergänge darstellen. Auf dieser Basis wurde formell nachgewiesen:

- Der Spin-Lock-Graph G_{SL} ist ein schwacher Subgraph von G_{ML} (Satz 2).
- Das binäre Semaphore G_{BS} ist sowohl Subgraph von G_{ML} als auch von $G_{\text{ZS}}^{(k)}$ für alle $k \geq 1$ (Sätze 3 und 5).
- Der Mutex-Lock G_{ML} ist Subgraph der Bedingungsvariable G_{CV} (Satz 4).
- Die vollständige Hierarchie ist in Korollar 1 zusammengefasst.
- Der Subgraph Algorithmus erkennt alle diese Beziehungen korrekt und effizient in $O(n^3)$ (Satz 1).

7.2 Ausblick

Die vorliegenden Ergebnisse eröffnen eine Vielzahl weiterführender Forschungsrichtungen, die im Folgenden detailliert ausgeführt werden.

7.2.1 Erweiterung auf gewichtete und probabilistische Graphen

Die derzeitige Modellierung verwendet ungewichtete Digraphen. Eine natürliche Erweiterung besteht darin, Kantengewichte einzuführen, die Ausführungswahrscheinlichkeiten oder mittlere Wartezeiten kodieren. Beispielsweise könnten die Kanten eines Mutex-Lock-Graphen mit empirisch gemessenen Transitionswahrscheinlichkeiten aus Linux-Kernel-Traces annotiert werden [5]. Für gewichtete Subgraph-Erkennung müsste die LCS-Komponente des Subgraph Algorithmus durch ein gewichtetes Sequenzalignment (ähnlich dem Smith-Waterman-Algorithmus [6]) ersetzt werden. Dies würde eine probabilistische Subgraph-Relation ermöglichen, bei der eine Einbettung als gültig gilt, wenn das gewichtete Alignment einen vorgegebenen Schwellwert überschreitet – nützlich für die Leistungsanalyse realer Betriebssystemkerne.

7.2.2 Parallele und verteilte Berechnung des Subgraph Algorithmus

Die n Rotationsschritte des Subgraph Algorithmus sind gemäß Lemma ?? voneinander unabhängig. Dies ermöglicht eine *embarrassingly parallel* Implementierung auf Mehrkernprozessoren: Jeder der n Rotationsschritte wird einem separaten CPU-Kern zugewiesen; die n LCS-Berechnungen laufen simultan. Die Gesamtlaufzeit sinkt von $O(n^3)$ auf $O(n^2)$ bei n verfügbaren Kernen [7]. Darüber hinaus lässt sich die DP-Matrix jeder LCS-Berechnung auf SIMD-fähige Vektorregister (AVX-512) abbilden, was die Berechnungslatenz pro Rotation um einen weiteren konstanten Faktor reduziert [8]. Für extrem große Graphen ($n > 10^4$) ist eine GPU-basierte Implementierung (CUDA/OpenCL) denkbar, bei der jede Zeile der DP-Tabelle als separater CUDA-Thread verarbeitet wird.

7.2.3 Anwendung auf formale Verifikation von Synchronisationsprotokollen

In der formalen Methodik werden Synchronisationsprotokolle häufig als Kripke-Strukturen oder Transitionssysteme modelliert und mittels Model-Checking auf Eigenschaften wie Deadlock-Freiheit und starvation-Freiheit überprüft [9]. Der Subgraph Algorithmus kann hier als *Vorstufe* eingesetzt werden: Bevor ein teures Model-Checking durchgeführt wird, prüft er, ob zwei Synchronisationsprotokolle strukturell verwandt sind. Falls $G_A \subseteq G_B$ gilt und G_B bereits als deadlock-frei verifiziert wurde, lässt sich diese Eigenschaft unter zusätzlichen Monotonie-Bedingungen auf G_A übertragen – eine Technik, die als *compositional reasoning* [10] bekannt ist.

7.2.4 Integration in statische Analysetools und Compiler

Moderne Compiler (GCC, LLVM/Clang) erzeugen interne Darstellungen von nebenläufigem Code, die als Kontrollflussgraphen (CFG) oder Threaded-Code-Graphen vorliegen [11]. Der Subgraph Algorithmus könnte als Compiler-Pass implementiert werden, der erkennt, wenn ein Programmierer einen Mutex-Lock verwendet, obwohl ein einfacher Spin-Lock ausreichen würde – oder umgekehrt, wenn ein Spin-Lock einen Mutex ersetzen sollte (z. B. bei langen kritischen Regionen). Diese automatische Empfehlung würde die Systemleistung

verbessern, ohne dass Programmierer selbst fundierte Kenntnisse aller Synchronisationsmechanismen benötigen.

7.2.5 Erweiterung auf hierarchische Parallelität (OpenMP, MPI)

Die hier betrachteten Synchronisationsmechanismen beziehen sich hauptsächlich auf POSIX-Threads (Pthreads) innerhalb eines Prozesses. In modernen Hochleistungsrechnersystemen spielen jedoch auch Multi-Ebenen-Parallelitätsmodelle wie OpenMP [7] (für Shared-Memory-Parallelismus) und MPI [12] (für Distributed-Memory-Parallelismus) eine wesentliche Rolle. Die Zustandsgraphen von MPI-Kollektiv-Operationen (z. B. `MPI_Barrier`, `MPI_Reduce`) könnten analog zu den hier definierten Synchronisationsgraphen modelliert und mittels Subgraph Algorithmus klassifiziert werden. Eine Hierarchie $G_{\text{Barrier}} \subseteq G_{\text{AllReduce}}$ würde beispielsweise belegen, dass eine Barriere strukturell in einer AllReduce-Operation enthalten ist – mit direkten Implikationen für die Optimierung kollektiver MPI-Kommunikation.

7.2.6 Maschinelles Lernen zur Subgraph-Vorhersage

Graph Neural Networks (GNNs) [13] wurden in den letzten Jahren erfolgreich für Subgraph-Erkennungsaufgaben eingesetzt. Eine interessante Forschungsrichtung besteht darin, den Subgraph Algorithmus als *Label* für das Training eines GNN zu verwenden: Für eine große Datenbank von Synchronisationsgraphen werden die Subgraph-Beziehungen mittels Subgraph Algorithmus vorberechnet; das GNN lernt anschließend, diese Beziehungen in $O(1)$ vorherzusagen (nach dem Training). Dieser Ansatz kombiniert die Korrektheit des exakten Algorithmus mit der Schnelligkeit eines gelernten Modells – besonders interessant für Echtzeit-Anwendungen wie die Live-Analyse laufender Betriebssystemprozesse.

7.2.7 Erweiterung auf hybride Synchronisationsprotokolle

In der Praxis verwenden Betriebssysteme hybride Protokolle, die mehrere Mechanismen kombinieren: Der Linux-Kernel nutzt z. B. `futex` (Fast User-space Mutex), der im Unstrittenen Fall im Userspace operiert und nur bei Konkurrenz in den Kernel wechselt [14]. Der Zustandsgraph eines Futex besitzt daher Teile des Spin-Lock-Graphen (für den streitfreien Fall) und Teile des Mutex-Lock-Graphen (für den strittigen Fall). Eine Erweiterung des Subgraph Algorithmus auf *zusammengesetzte* Graphen, die als Vereinigung von Teilgraphen verschiedener Mechanismen modelliert werden, wäre ein wertvoller Beitrag. Dies erfordert eine Anpassung der Signaturberechnung auf Teilgraph-Mengen und eine modifizierte LCS-Variante, die Teilsequenzen aus mehreren Quellgraphen berücksichtigt.

7.2.8 Anwendung auf Echtzeitsysteme und AUTOSAR

In eingebetteten Echtzeitsystemen (RTOS) und dem AUTOSAR-Standard [15] spielen Synchronisationsmechanismen eine sicherheitskritische Rolle. Prioritätsinversion – das bekannte Problem, bei dem ein niedrigpriorer Prozess einen hochprioreren blockiert – ist direkt als Subgraph-Eigenschaft formulierbar: Tritt der Subgraph $G_{\text{priorityInversion}} \subseteq G_{\text{Protokoll}}$ auf, ist das Protokoll anfällig für dieses Problem. Die Anwendung des Subgraph Algorithmus auf AUTOSAR-Kommunikationsprotokolle könnte automatisch solche sicherheitskritischen Muster erkennen, bevor ein Steuergerät in die Serienfertigung geht [16].

7.2.9 Quantencomputing und Quanten-Synchronisation

Mit dem Aufkommen von Quantencomputern entstehen neue Parallelitätsparadigmen, in denen klassische Konzepte wie Mutex-Locks und Semaphoren nicht direkt übertragbar sind [17]. Dennoch existieren Ansätze für Quanten-Zustandsautomaten, die ähnliche Strukturen wie die hier modellierten Synchronisationsgraphen aufweisen. Eine Forschungsfrage ist, ob der Subgraph Algorithmus auf Quanten-Zustandsgraphen – modelliert als unitäre Transformationen auf Hilbert-Räumen – erweitert werden kann. Dies würde erfordern, die diskrete Signatur-Funktion durch eine quantenmechanische Observable zu ersetzen und die LCS-Berechnung auf Quanten-Parallelismus zu verallgemeinern.

7.2.10 Offene Komplexitätsfragen

Aus theoretischer Sicht bleibt die Frage offen, ob eine *unbedingte* untere Schranke von $\Omega(n^3)$ ohne die SETH-Hypothese beweisbar ist. Aktuelle Ergebnisse [18] zeigen lediglich eine bedingte Schranke unter SETH. Eine Auflösung dieser Frage wäre ein bedeutender Fortschritt in der Komplexitätstheorie des Subgraph-Isomorphismus. Darüber hinaus ist unklar, ob der Subgraph Algorithmus auf *approximative* Subgraph-Beziehungen (LCS $\geq \alpha \cdot n$ für ein festes $\alpha \in (0, 1)$) in besserer Laufzeit als $O(n^3)$ verallgemeinert werden kann, ohne SETH zu widerlegen.

References

- [1] S. Epp, *Der Subgraph Algorithmus*, Technischer Bericht, Bielefeld, Januar 2026. <https://gitlab.com/epp-group/subgraph>
- [2] A. S. Tanenbaum und H. Bos, *Modern Operating Systems*, 4. Auflage, Pearson, 2014.
- [3] A. Silberschatz, P. B. Galvin und G. Gagne, *Operating System Concepts*, 10. Auflage, Wiley, 2018.
- [4] M. Herlihy und N. Shavit, *The Art of Multiprocessor Programming*, Morgan Kaufmann, 2008.
- [5] R. Love, *Linux Kernel Development*, 3. Auflage, Addison-Wesley, 2010.
- [6] T. F. Smith und M. S. Waterman, “Identification of Common Molecular Subsequences”, *Journal of Molecular Biology*, Bd. 147, S. 195–197, 1981.
- [7] OpenMP Architecture Review Board, *OpenMP Application Program Interface Version 3.1*, 2011. <https://www.openmp.org>
- [8] T. Rognes, Faster Smith-Waterman Database Searches with Inter-Sequence SIMD Parallelisation, *BMC Bioinformatics*, 12:221, 2011.
- [9] E. M. Clarke, O. Grumberg und D. Peled, *Model Checking*, MIT Press, 1999.
- [10] K. L. McMillan, *Symbolic Model Checking*, Kluwer Academic Publishers, 1993.
- [11] C. Lattner und V. Adve, LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation, *Proc. CGO 2004*, S. 75–86.

- [12] MPI Forum, *MPI: A Message-Passing Interface Standard, Version 2.2*, 2009. <https://www.mpi-forum.org>
- [13] F. Scarselli et al., “The Graph Neural Network Model”, *IEEE Transactions on Neural Networks*, Bd. 20, Nr. 1, S. 61–80, 2009.
- [14] H. Franke, R. Russell und M. Kirkwood, “Fuss, Futexes and Furwocks: Fast Userland Locking in Linux”, *Proc. Ottawa Linux Symposium*, S. 479–495, 2002.
- [15] AUTOSAR Consortium, *AUTOSAR Classic Platform – Release 4.4.0*, 2021. <https://www.autosar.org>
- [16] ISO, *ISO 26262: Road Vehicles – Functional Safety*, 2nd edition, 2018.
- [17] S. J. Gay, “Quantum Programming Languages: Survey and Bibliography”, *Mathematical Structures in Computer Science*, Bd. 16, Nr. 4, S. 581–600, 2006.
- [18] A. Abboud, A. Backurs und V. V. Williams, Tight Hardness Results for LCS and Other Sequence Similarity Measures, *Proc. FOCS 2015*, S. 59–78.